DESARROLLO BACKEND

Backend · Estado y API

Marcos Teixido · German Velazquez · Axel Nievas — 12 de octubre de 2026

La Fase 1 está terminada y el modo vs. Stockfish (Fase 2) funciona: se puede jugar 1v1 y contra Stockfish por la API. Falta que el frontend la consuma, definir la base de datos real y el servicio Python de la IA propia.

Fase actual
Desarrollo. Fase 1 (motor de reglas y juego local) terminada; Fase 2 (Stockfish) funcionando; Fase 3 (IA propia) conectada del lado del backend, esperando el servicio de Simulación.

Alcance: lo que Backend no tocó
Backend no modificó el trabajo de otros grupos. **IA propia:** no se desarrolló nada; solo la llamada al servicio con el contrato de Diseño Técnico. **Modelo de datos:** el de Modelado de Datos no se cambió; Backend propone un script de base para poder funcionar y Datos decide. **Stockfish:** Backend escribió solo un adaptador mínimo para poder probar, con lo que ya definía Arquitectura (6.1); el motor es de Simulación, que decide si lo adopta o lo reemplaza por el suyo.

Qué se hizo esta fase

- API REST en **Node.js + Express 5** (pnpm), siguiendo los endpoints y clases de Diseño Técnico: GestorDePartida, ServicioHistorial, Tablero, ValidadorDeJugada y el patrón Strategy para los oponentes (GameSessionFactory).
- **Reglas completas** con chess.js: jugadas legales, jaque, enroque, captura al paso, promoción a elección, jaque mate, ahogado, triple repetición, regla de los 50 movimientos, material insuficiente y abandono.
- **Persistencia jugada a jugada** en PostgreSQL: la jugada, el nuevo FEN y el cierre de la partida se guardan en una sola transacción. Pausa, reanudación, historial con filtros y exportación PGN.
- **vs. Jugador** funcionando. **vs. Stockfish** funcionando con un adaptador mínimo de prueba (proceso hijo + UCI, Skill Level 3/10/20, timeout de 3 s con reintento), a disposición de Simulación. Probado con Stockfish 17.1 en Windows.
- **vs. IA propia**: solo el lado del backend (la llamada al servicio Python y, si no conoce la posición, Stockfish nivel 0). La IA es de Simulación y no se tocó.
- Propuesta de script de base para Modelado de Datos (su modelo no se modificó) y documento de observaciones al diseño técnico (14 puntos).
- Ajuste para el frontend (Vite, puerto 5173): CORS habilitado. Guía de conexión con las 16 tareas del front.

Qué falta

- Que **Frontend** consuma la API (crear partida, tablero desde el FEN, jugadas, turno de máquina, diálogos, historial).
- Definir con GC y Modelado de Datos la **base real** (PostgreSQL local o Supabase) y correr la migración.
- **Servicio Python de la IA** propia (Simulación).
- **Importar PGN** (corrección C9 del profesor): falta el caso de uso (Requisitos) y el endpoint (Diseño Técnico).
- Subir el código al repositorio en la rama `feature/backend-fase1` y abrir el pull request para el Tech Lead.

Bloqueos y dudas abiertas

- ¿La dificultad va por **ELO o por nivel**? Consultar al profesor. La tabla de perfiles ya tiene los dos datos.
- **Endpoints y puerto** distintos entre Arquitectura (`/api`, 4000, Next.js) y Diseño Técnico (sin `/api`, 3001). Seguimos a Diseño Técnico; el front real es Vite en 5173.
- Base local o **Supabase**: la guía de despliegue tiene la contraseña escrita.
- **Reloj** de partida: el front muestra 15:00 pero ningún requisito pide control de tiempo.

Endpoints disponibles

Dirección local: `http://localhost:3001`. Todas las respuestas son JSON; los errores tienen la forma `{ "error": { "code", "message", "details"? } }`.

Método	Ruta	CU	Qué hace
GET	/health	–	Estado del servidor y de la base
GET	/difficulty-profiles	CU-01	Fácil, Medio y Difícil por motor, con rango ELO
POST	/games	CU-01	Crea la partida en curso
GET	/games	CU-05	Historial: filtros mode, difficulty, status, result, alias
GET	/games/:id	CU-05	Estado actual + lista de jugadas
GET	/games/:id/pgn	CU-05	Exporta la partida en PGN
GET	/games/:id/legal-moves?square=e2	CU-06	Destinos legales de una pieza
POST	/games/:id/moves	CU-06	Jugada del humano { from, to, promotion? }
POST	/games/:id/engine-move	CU-03/04	Jugada de la máquina (Stockfish o IA)
POST	/games/:id/pause · /resume	CU-02..05	Guardar y salir · reanudar
POST	/games/:id/resign	RN-03	Abandono { color }

HTTP	code	Cuándo
400	VALIDATION_ERROR	Datos mal formados (falta alias, casilla inválida, etc.)
404	NOT_FOUND	La partida no existe
409	INVALID_STATE · NOT_YOUR_TURN	Partida pausada o finalizada · turno equivocado
422	ILLEGAL_MOVE · PROMOTION_REQUIRED	Jugada ilegal · falta elegir la pieza de coronación
502 / 503 / 504	ENGINE_BAD_MOVE · ENGINE_UNAVAILABLE · ENGINE_TIMEOUT	Problema del motor; se puede reintentar

Pruebas

Tipo	Cantidad	Qué cubren
Reglas de ajedrez (Tablero)	15	Cada regla de RN-02 Bis y RN-03 con posiciones concretas.
Validación de entrada	22	Una prueba por cada entrada válida e inválida de la API; configuración y CORS.

Tipo	Cantidad	Qué cubren
API por caso de uso	25	CU-01 a CU-06 completos: partida hasta el mate, pausa, abandono, turno de máquina, timeout, respaldo de la IA, historial, PGN y CORS.
Adaptador de Stockfish	11	Secuencia UCI, cola, timeout, caída y reinicio del motor; una con Stockfish 17.1 real.
Integración con PostgreSQL	7	Migración, persistencia real, vista de entrenamiento y restricciones de la base.

80 pruebas, todas pasan (`pnpm test`, `pnpm test:integration`).

Cómo levantarlo

`pnpm install` · `pnpm test` · `pnpm dev:memory` (sin base de datos, en localhost:3001). Para jugar contra Stockfish, poner `STOCKFISH_PATH` en el `.env` con la ruta al ejecutable. Con base real: completar `DATABASE_URL`, `pnpm migrate` y `pnpm dev`.

Documentos relacionados: Cambios al modelo de datos · Integración con Stockfish · Qué falta de la IA propia · Observaciones al diseño técnico · Conexión Front-Back · `backend/README.md`.